

Scalable Tokio Timer

Qiqi Zhang

Timers Are Everywhere In Async App

- Every Sleep
- Every Timeout
- Every Retry
- ...

Timer Looks Simple

You schedule something in the future,
and the runtime wakes it up.

Timeout In Sync Rust

```
do_something();  
if instant.elapsed() >= TIMEOUT {  
    return Err(...);  
}  
do_something();
```

Timeout In Sync Rust

```
fn do_something() {  
    while instant.elapsed() >= TIMEOUT {  
        do_one_step();  
    }  
}  
do_something();
```

Timeout In Sync Rust

```
// thread 1  
sleep(30); // 30s  
pthread_cancel();  
  
// thread 2  
pthread_testcancel();
```

What's The Problem?

You have to do intrusive change
for all functions.

Async Rust (no intrusive changes)

```
let res_fut: Timeout = tokio::time::timeout(TIMEOUT, do_something_async());
impl Future for Timeout {
    fn poll() {
        if self.timer().elapsed(0 >= TIMEOUT) { /* abort */ }
        poll_do_something()
    }
}
```


What's At Cost?

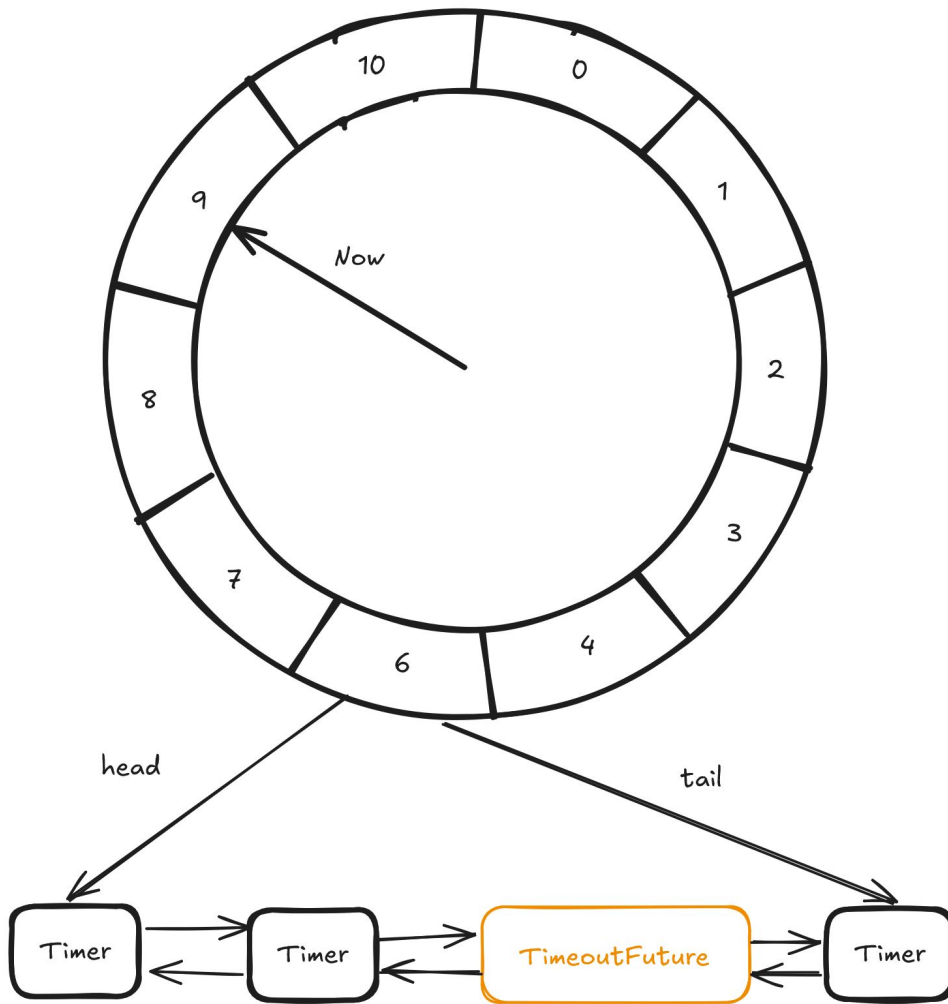
Timers are no longer local.

Double-Edged Sword

Async Rust enables flexibility,
but introduces structural contentions.

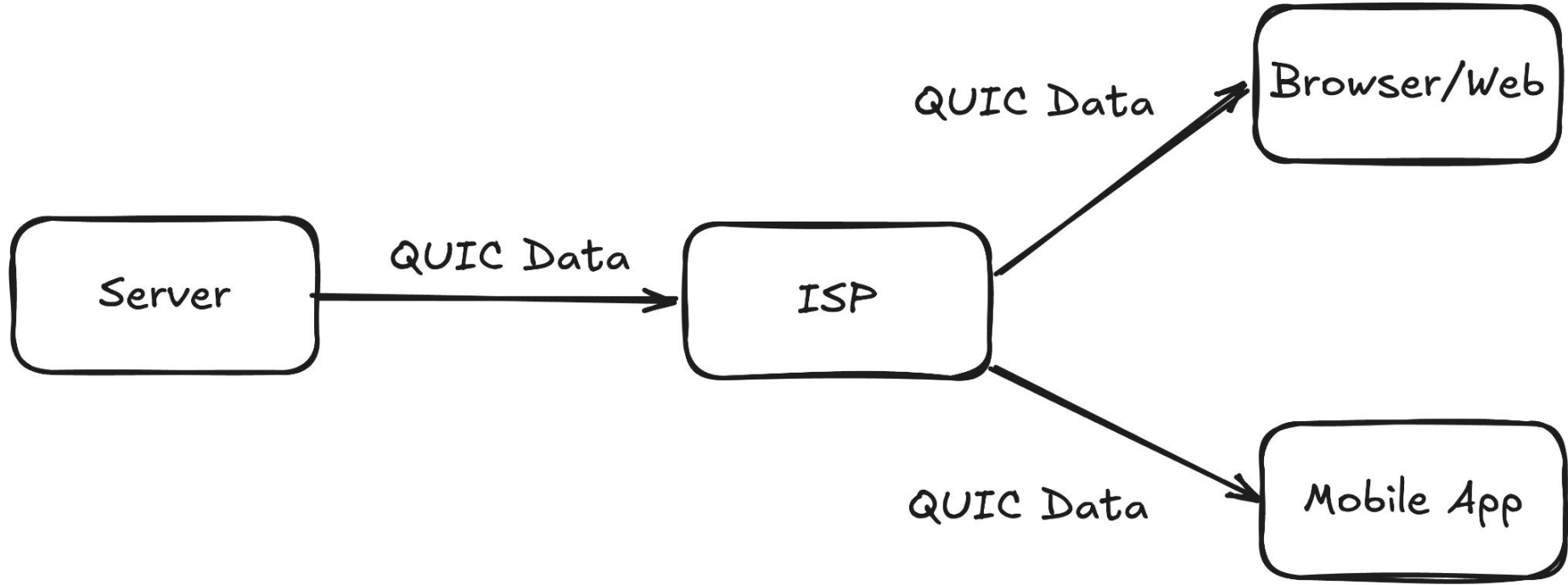
Agenda

- What's the default implementation of Tokio timer.
- Bottleneck shown by a use case.
- What have we achieved for optimizing the default implementation.
- Introduce the alternative implementation
 - What's the initial implementation, why it failed.
 - Inject Queue
 - Final implementation
- Performance improvement for above use case.

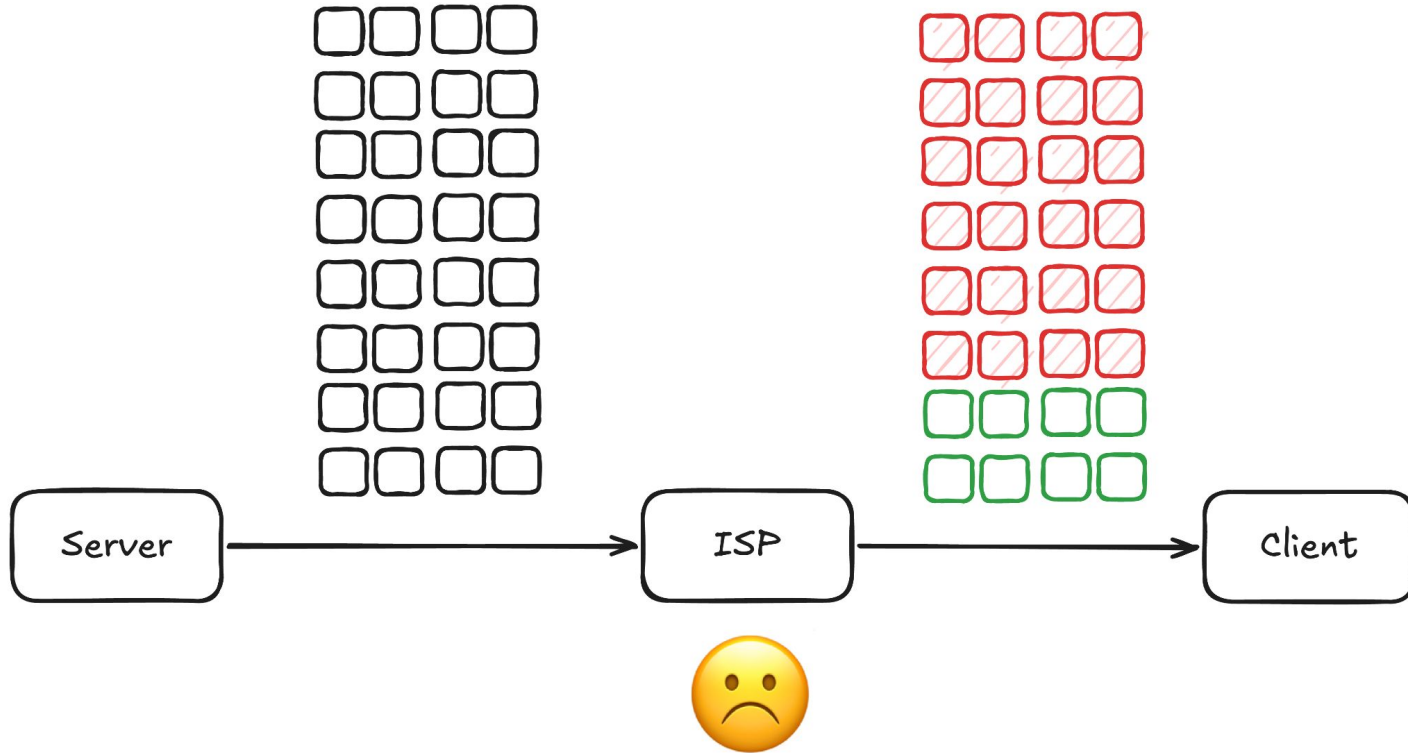


Default
Implementation

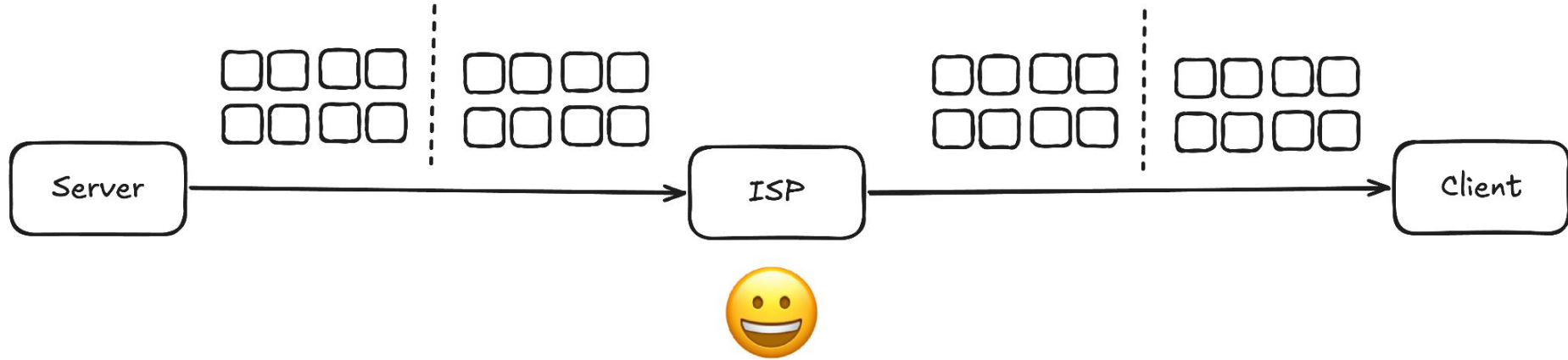
Use Case: UDP Flow Control



Use Case: UDP Flow Control

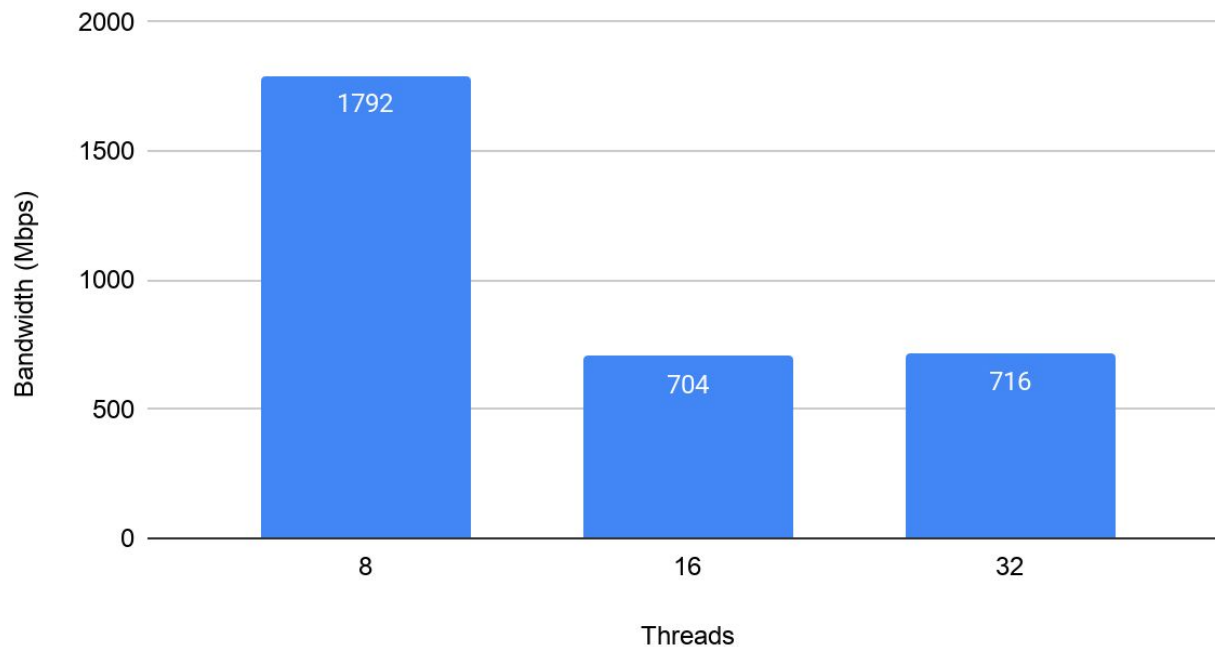


Use Case: UDP Flow Control

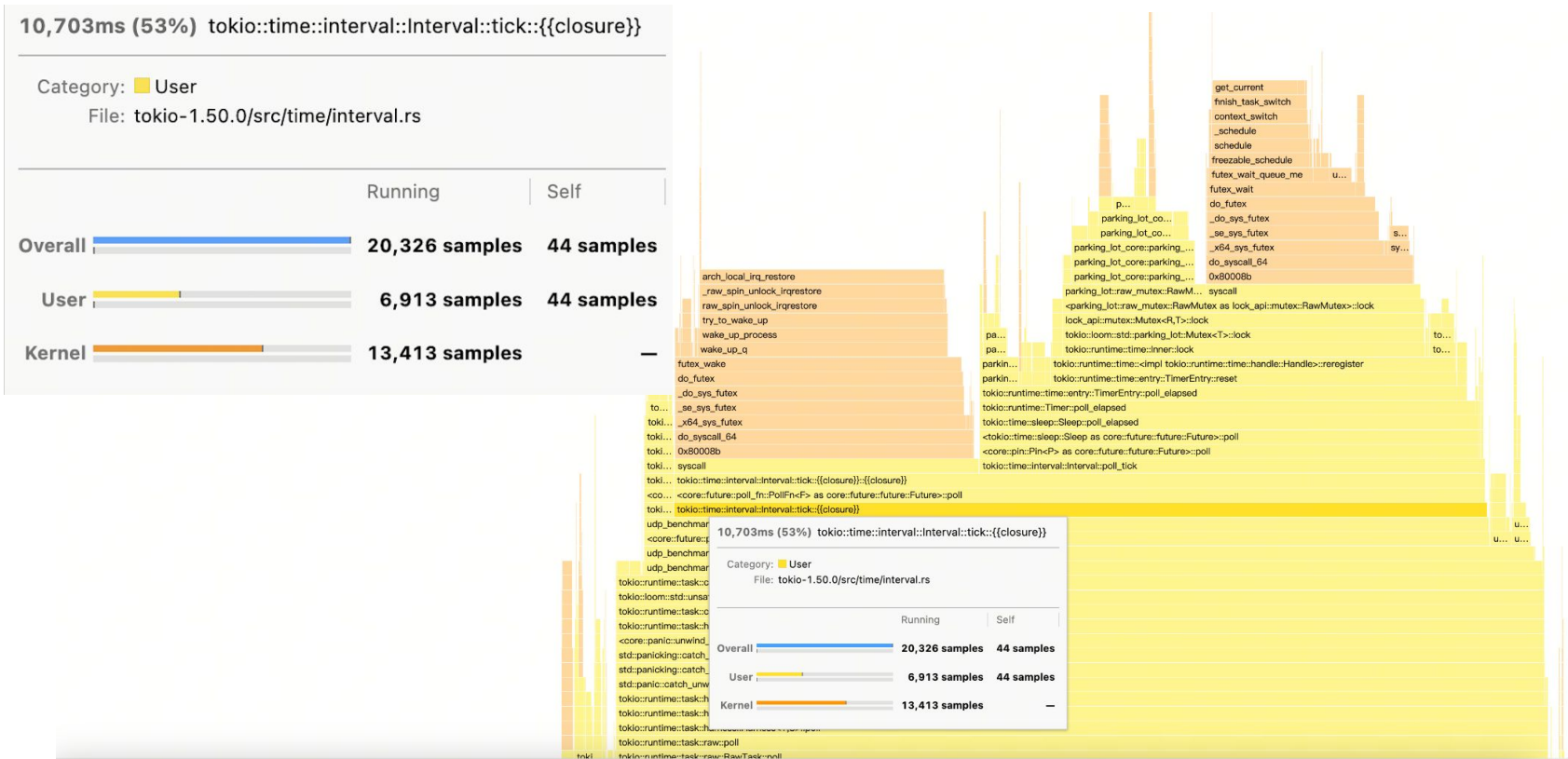


Use Case: UDP Flow Control

Bandwidth (Mbps per Thread) vs. Threads



Use Case: UDP Flow Control



Reduce contention in timer driver #6504

Open



Darksonn opened on Apr 21, 2024 · edited by mox692

Edits

Member



I have often heard reports about performance issues due to large amounts of timers. Specifically, this is due to contention on the mutex in the timer driver.

Some ideas:

- If a `Sleep` instance is dropped before it is polled, its destructor will still lock the mutex to check whether it needs to be removed from the driver. This seems unnecessary, and will be a common case for timeouts.
- We could shard the timer into multiple instances with one per runtime thread.
- It's worth thinking about whether our timers are optimized for the case where most timers complete normally (often the case of `tokio::time::sleep`), or the case where most timers are cancelled before they complete (often the case for timeouts). Can we do anything to differentiate this in the driver?
- Perhaps a completely different implementation is appropriate?

Before spending a lot of time on a large change, please check with us beforehand by posting here or in Discord so that we can discuss whether your implementation idea is what we want.

PRs related to this issue:

- ✓ Lazily init timers on first poll ([#6512](#))
- ✓ Using sharded locks instead of a global lock for Timers ([#6534](#))
- ✓ Avoid traversing entries in the time wheel twice ([#6584](#))

Create sub-issue



6



Darksonn added

E-help-wanted

A-tokio

M-runtime

M-time

E-hard

T-performance

on Apr 21, 2024

Lazy Initialization

TimeoutFuture

- * expired_at
- * waker

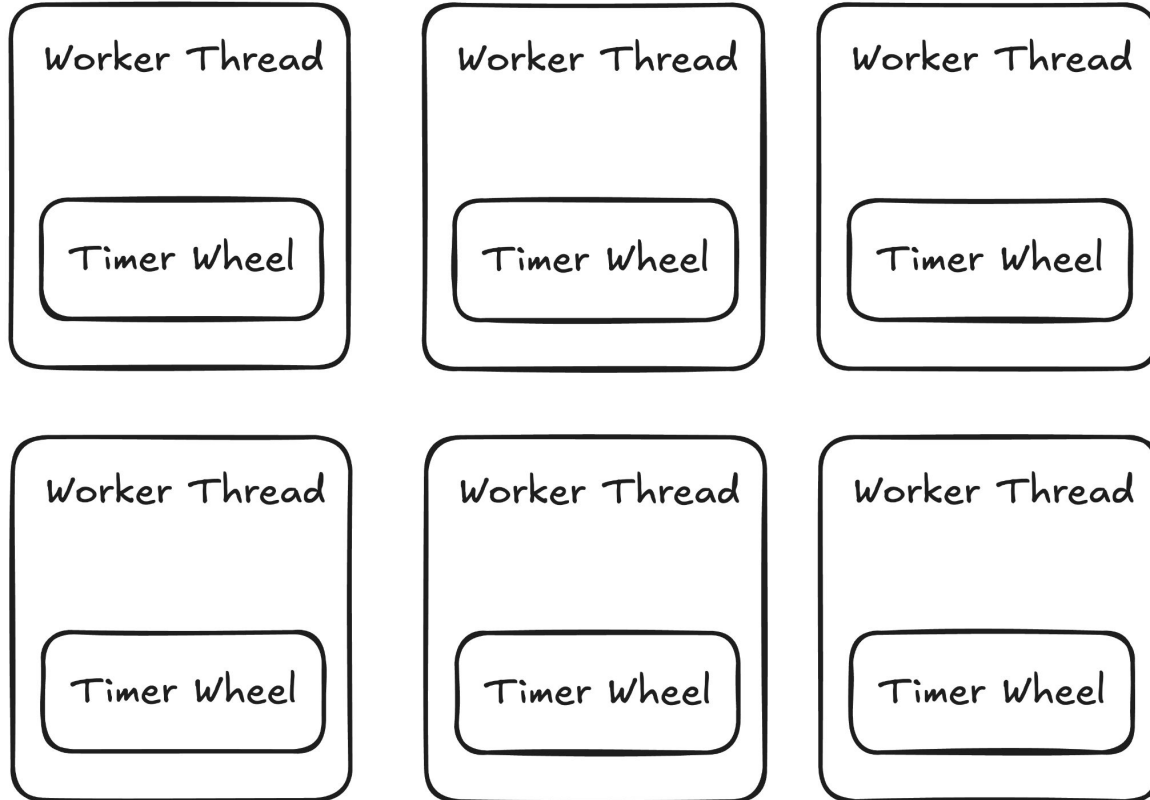
Lazy Initialization

TimeoutFuture

* expired_at

* Option<Waker>

Sharding Timer Wheels



Sharding Timer Wheels

```
loop {  
    let evs = epoll_wait(...);  
    for ev in evs {  
        ev.wake_future();  
    }  
  
    for shard in self.timer_shards {  
        let wheel = shard.lock();  
        wheel.process();  
    }  
}
```

How Does Kernel Handle It?

- Per-CPU timer wheel.
- Each wheel is protected by a lock.
- Lock the wheel when necessary.

Why Doesn't The Kernel Have This Problem?

- Each wheel is protected by **spin lock**.
- Scheduler is **disabled** when using spin lock.

RFC: Delayed cancellation of timer #7384

✓ Closed

✓ 2 / 2



ADD-SP opened on Jun 4, 2025 · edited by ADD-SP

Edits ▾

Member

...

Summary

Please check out the [#6504](#) for initial context.

This design reduces the lock contention of the time module, this improves the scalability for the async server to read/write async sockets with timeout.

- Use a local wheel for each worker thread instead of a global wheel.
- Do not register the timer into the wheel on creation.
- Register timer into the local wheel on the first `poll`.
- Cancel a timer by removing it from the local wheel directly if this timer is canceled by local thread.
- Cancel a timer by sending the intrusive list entry using `std::sync::mpsc` to the worker thread that owns it if this timer is canceled by remote thread.

Key Challenge

1. The timer must valid until we remove it from local wheel.
2. The `TimeoutFuture` can be drop in other threads.
3. We cannot remove dead timers on dropping `TimeoutFuture`.

Key Challenge

1. The timer must valid until we remove it from local wheel.
2. The `TimeoutFuture` can be drop in other threads.
3. We cannot remove dead timers on dropping `TimeoutFuture`.

Cross-Thread Cancellation Queue

std::sync::mpsc?

Extra heap allocation

Intrusive MPSC?

Spin waiting is not ideal

Intrusive MPSC (never spin)?

Looks good?

Mutex<IntrusiveList>?

Looks good?

Intrusive MPSC vs. Mutex<IntrusiveList>

See [Appendix] for benchmark script.

- tokio 1.47.1: the latest (at that time) release of tokio that uses the global timer wheel.
- Intrusive MPSC: the implementation I just mentioned above.
- `Mutex<IntrusiveList>` : you know what is it.

Cores	tokio 1.47.1	Intrusive MPSC	<code>Mutex<IntrusiveList></code>
2	105k	109k	112k
4	214k	224k	223k
8	450k	458k	462k
16	882k	1025k	1000k
32	386k	1418k	1452k

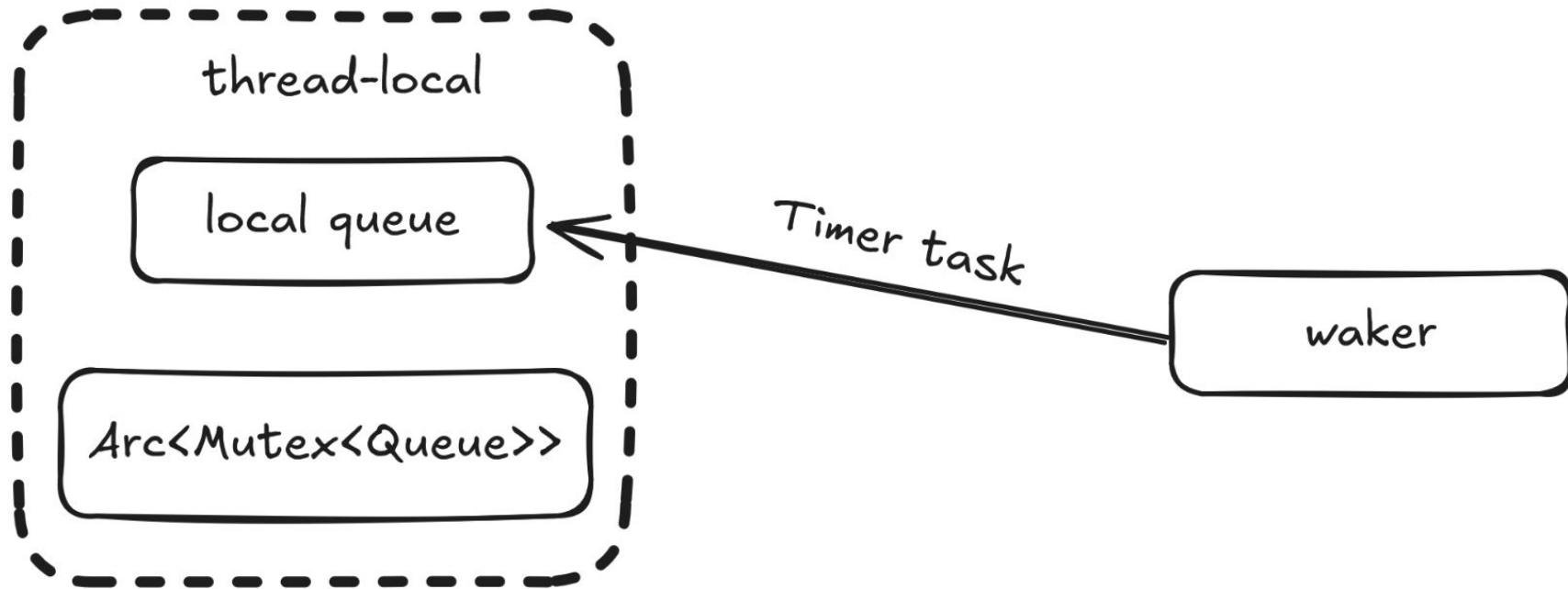
Mutex<IntrusiveList> Wins

For simpler implementation

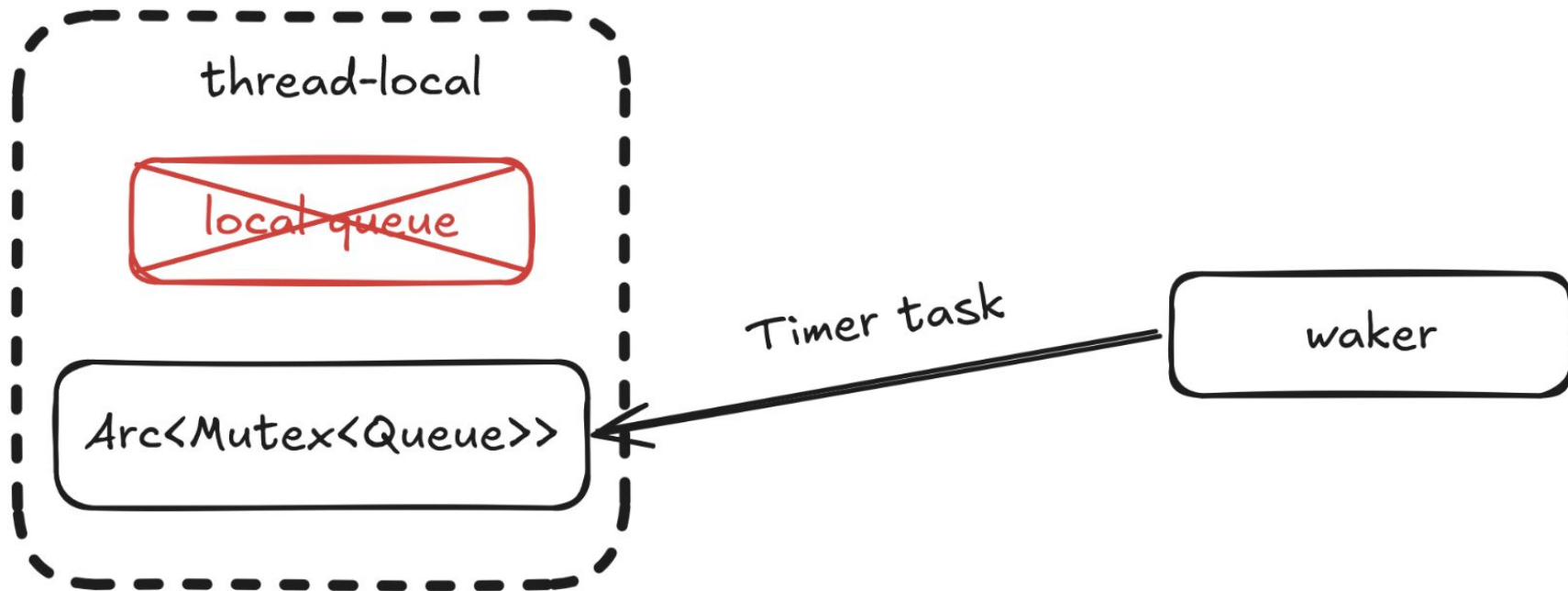
Inject Queue

When you wake up tasks
out of worker thread

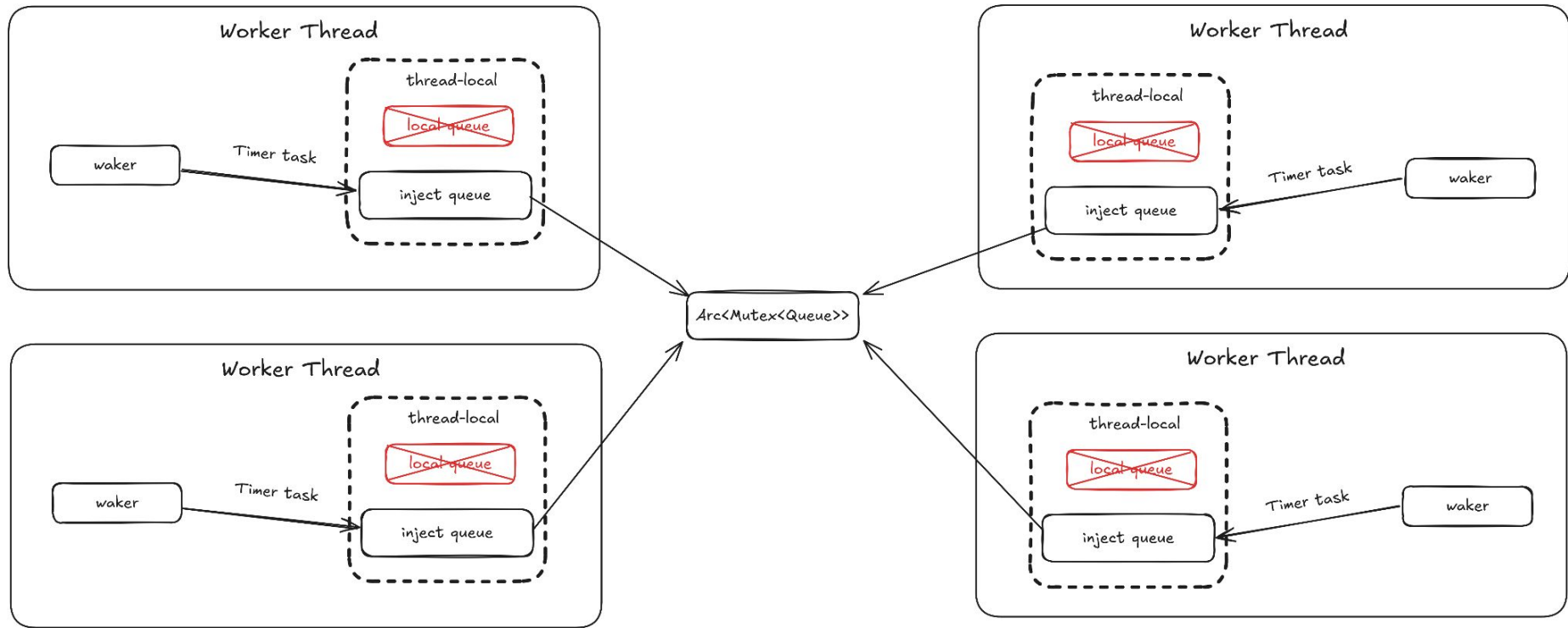
Inject Queue



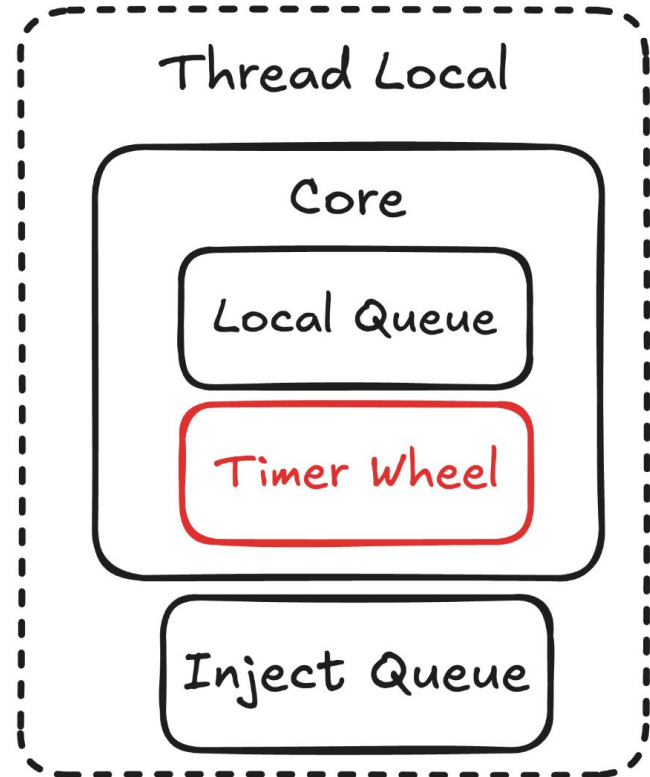
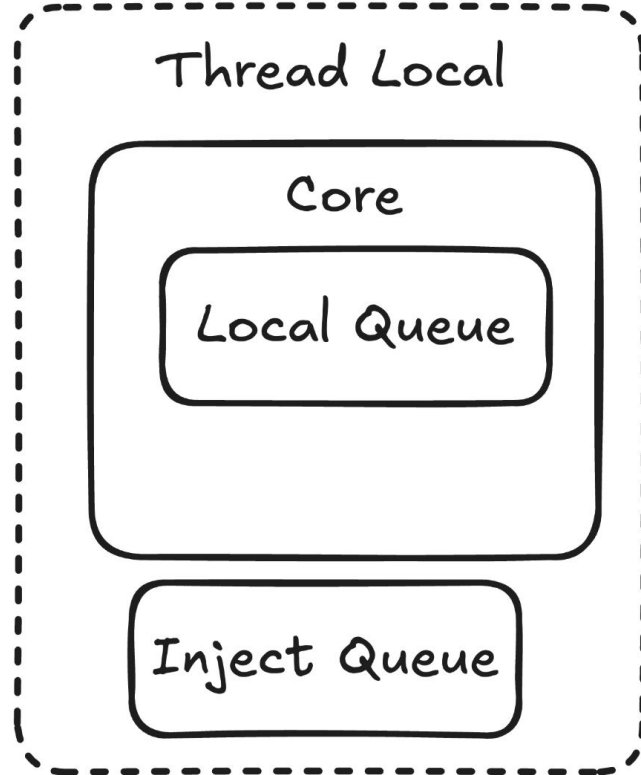
Inject Queue



Inject Queue



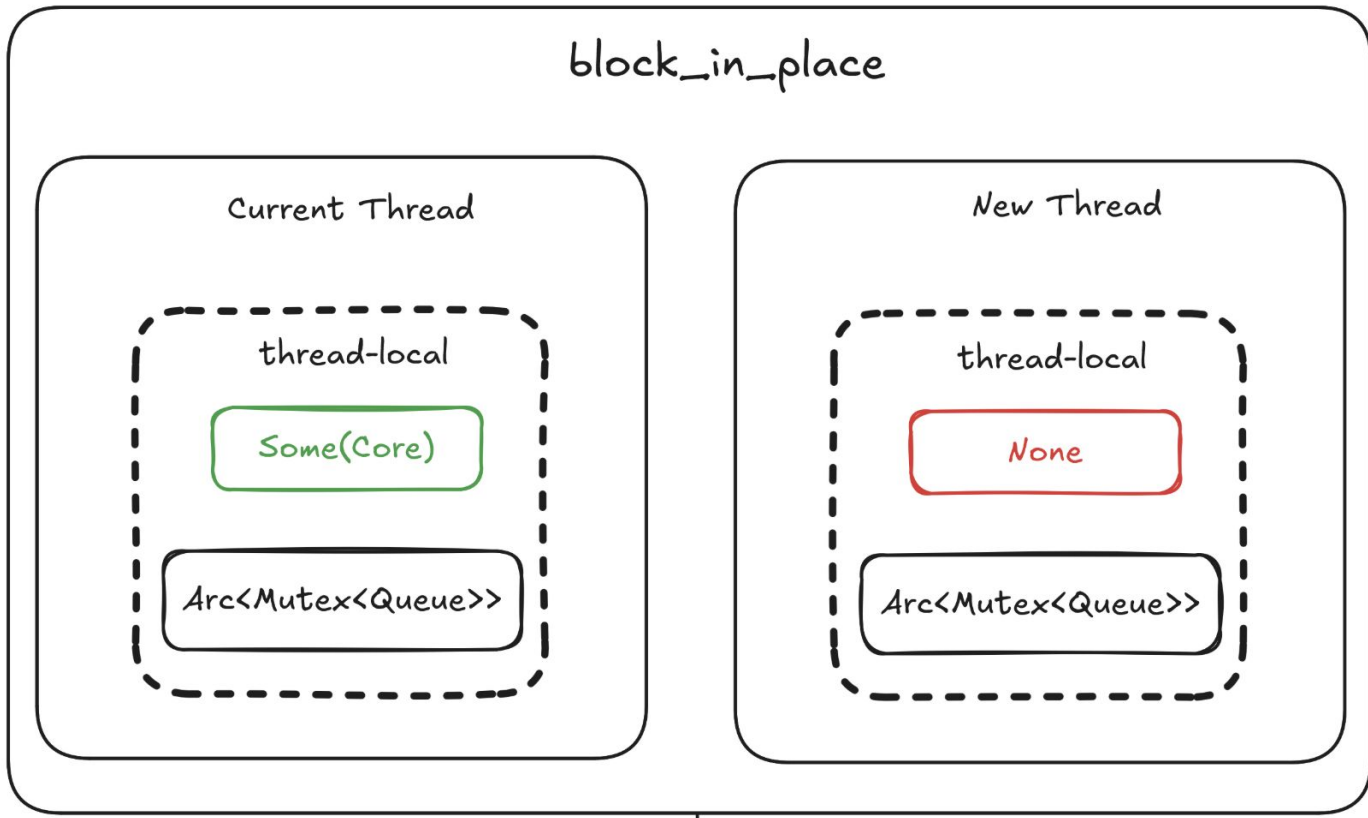
Inject Queue



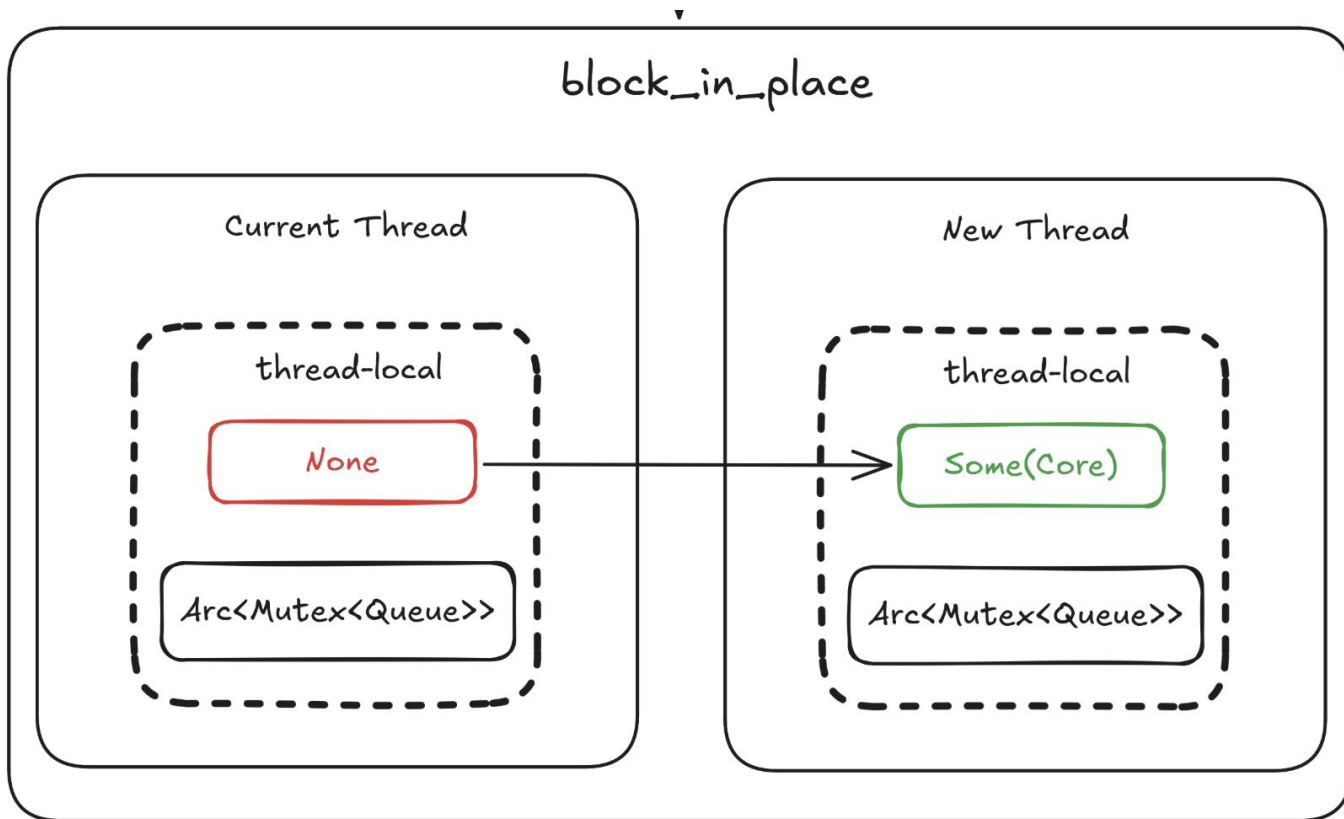
Option<Core>

It is **None**-able,
and it is actually **None** when waking up timers

Why is core **None**-able?



Why is core **None**-able?



One More Intrusive Queue

Just for collecting wakers

One More Intrusive Queue

Timer Wheel



Cancellation
Queue



One More Intrusive Queue

Timer Wheel



Cancellation Queue



Waker Queue

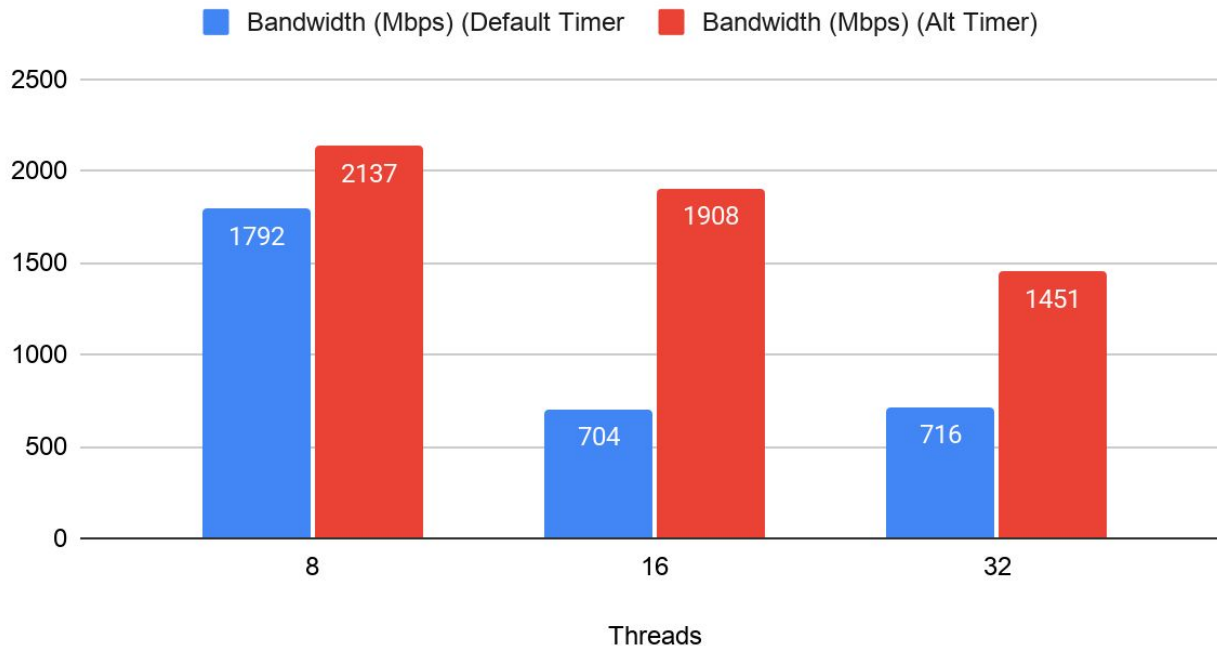


We Chose One More Intrusive Queue

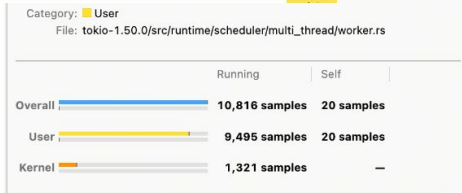
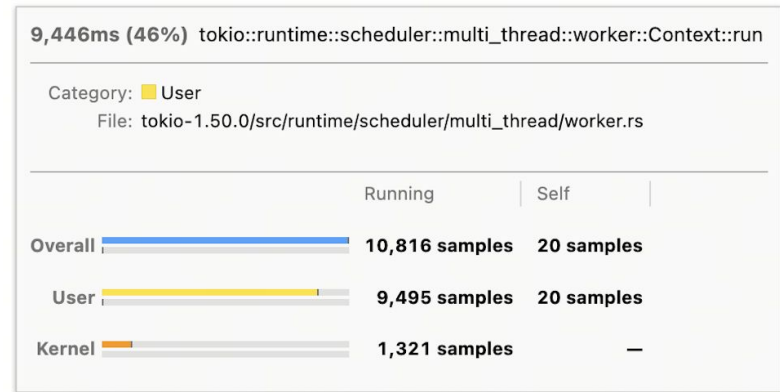
We accepted this complexity,
as we don't have a better choice at that time.

Use Case: UDP Flow Control

Bandwidth (Mbps per Thread) vs. Threads



Use Case: UDP Flow Control



Thanks



Alice Ryhl

Darksonn · she/her



Carl Lerche

carllerche

Runta – the infrastructure of agentic system

We are hiring!

qiqi@runta.com / hiring@runta.com